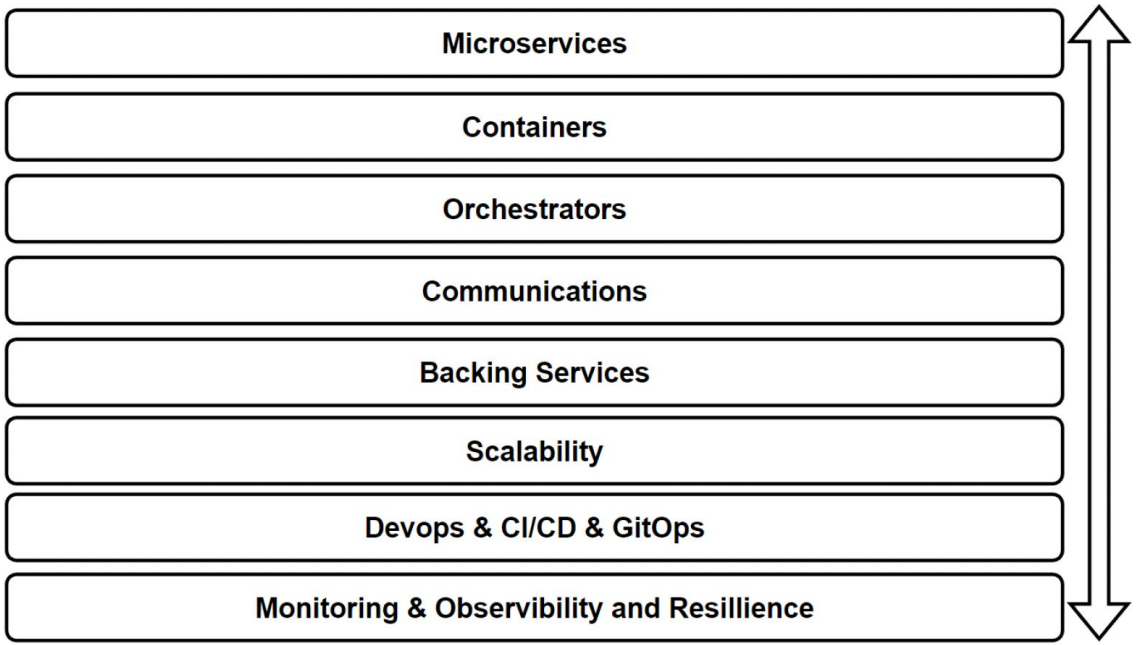
Week 9

- Cloud Native
- Docker
- DevOps

Cloud Native Pillars



Cloud Types - Private/On-premises, Public and Hybrid

Private/On-premises Cloud

- Owned and operated by a single organization
- Deployed within the organization's data center
- Increased privacy, security, and control
- Best for strict compliance, security, and data privacy requirements

Public Cloud

- Provided by third-party service providers (AWS, Azure, Google Cloud)
- Shared computing resources among multiple users
- Cost-effective, scalable, and flexible
- Providers manage infrastructure, maintenance, and updates

Hybrid Cloud

- Combination of private and public cloud infrastructures
- Leverages the benefits of both cloud types
- Enables workload movement between environments for efficiency and cost optimization.

Evolution of Cloud Platforms, Hosting Models

IaaS (Infrastructure as a Service)

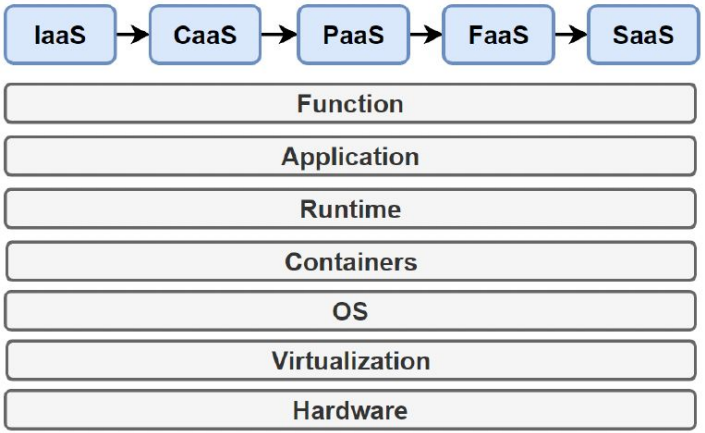
- Virtualized computing resources (VMs, storage, networking)
- Users control infrastructure, provider manages physical hardware
- Example: Amazon EC2, Microsoft Azure Virtual Machines
- Real-world: Hosting a web app on AWS EC2

CaaS (Container as a Service)

- Deploy, manage, and scale containerized applications
- Provider manages infrastructure and container orchestration
- Example: Google Kubernetes Engine, Amazon ECS, EKS, Fargate
- Real-world: Migrating a monolithic app to microservices using Google Kubernetes Engine

PaaS (Platform as a Service)

- Build, deploy, and manage applications without worrying about infrastructure
- Provider manages infrastructure, servers, networking, OS, etc.
- Example: Heroku, Microsoft Azure App Service
- Real-world: Developing a web app on Heroku



Contd.

FaaS (Function as a Service)

- Deploy individual functions, executed in response to events/triggers
- Provider manages infrastructure and auto-scales functions
- Example: AWS Lambda, Google Cloud Functions
- Real-world Example: Processing images using AWS Lambda

SaaS (Software as a Service)

- Software provided over the internet, eliminating need for installation
- Provider manages infrastructure, application, and data
- Example: Salesforce, Microsoft Office 365
- Real-world Example: Using Salesforce for CRM

Serverless

- Cloud provider manages infrastructure and auto-scales resources
- Pay only for resources consumed
- Example: Azure Functions with Azure Cosmos DB, AWS Lambda with Amazon DynamoDB
- FaaS is a specific implementation of serverless computing

Cloud Native Application Architecture

Designing, building, and running applications optimized for cloud computing environments.Scalable, resilient, and flexible applications for faster innovation and market responsiveness.

Microservices

- Small, loosely-coupled, independently deployable services.Improved agility, fault isolation, and continuous delivery.

Containers

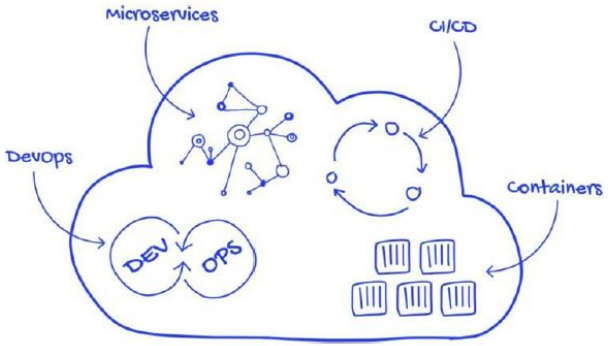
- Lightweight, portable runtime environments.Easier application management and deployment.Examples: Docker, containerd.

Orchestration

- Tools for managing containerized microservices.Automated scaling, rolling updates, and resource management.Example: Kubernetes.

Devops Practices

- Continuous Integration and Continuous Deployment (CI/CD), Infrastructure as Code (IaC).Streamlined development, testing, and deployment processes.



Contd.

Automation & CI/CD

- Infrastructure, deployment, scaling, and recovery processes.Uses CI/CD pipelines to automate the process of building, testing, and deploying code.

Scalability

- Horizontal scaling for optimal resource usage and performance.

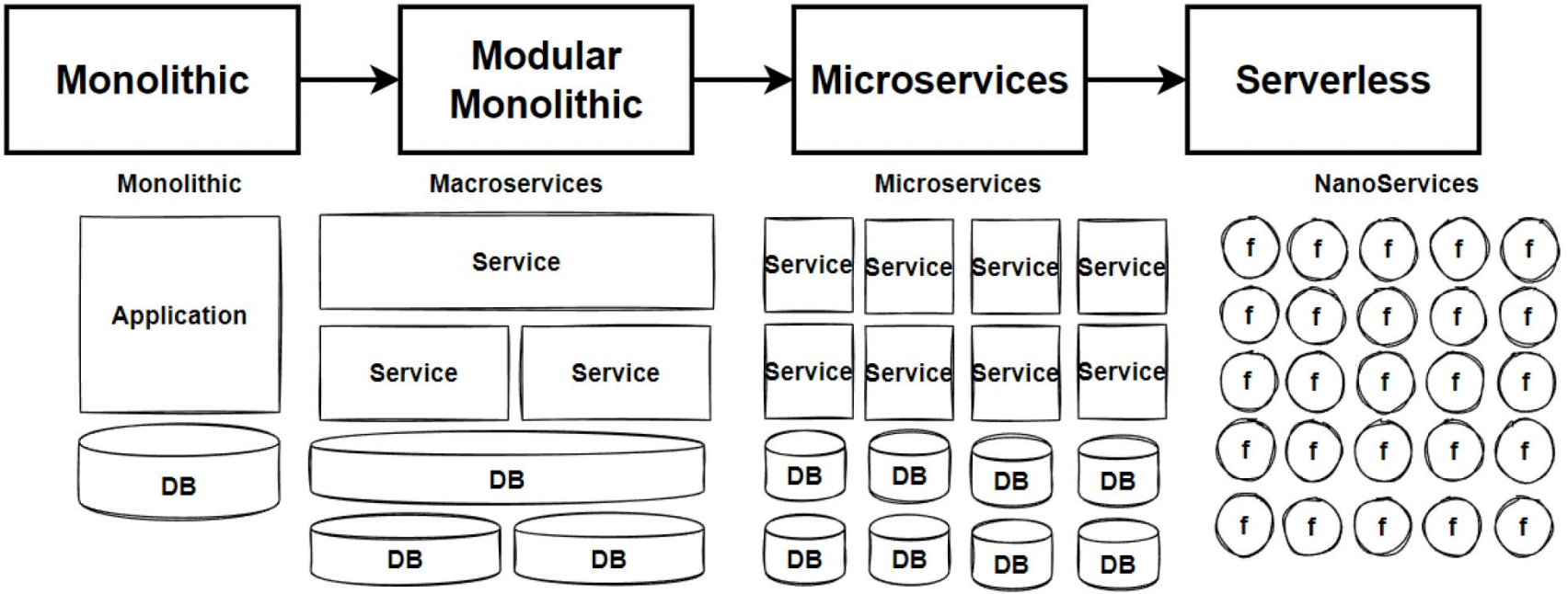
Resilience

- Fault-tolerant and self-healing.Techniques: circuit breakers, retries, timeouts.High availability and minimized downtime.

Observibility

- Built-in monitoring, logging, and tracing.Insights into performance, health, and behavior.Tools: Prometheus, Fluentd, Jaeger.

Architecture Design Journey



Cloud Path of Legacy Applications

Organizations move to the cloud for the agility and speed applications. Able to set up thousands of servers (VMs) in the cloud in minutes. No single, one-size-fits-all strategy for migrating apps to the cloud.

Level 1: Infrastructure-Ready Applications in the Cloud

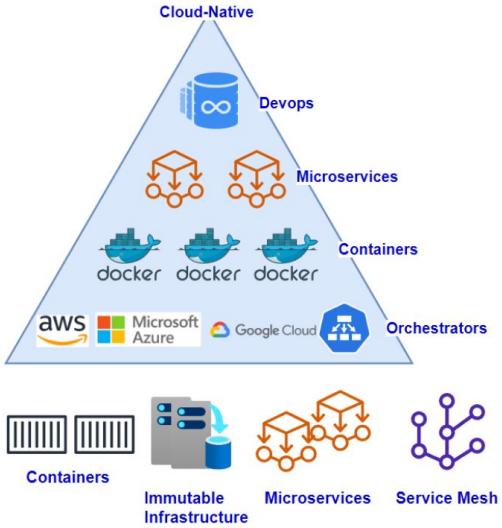
- Migrate or rehost existing on-premises applications to IaaS
- Minimal changes, retain original structure
- "Lift & Shift"

Level 2 -Cloud-Enhanced Applications

- Use modern cloud technologies (containers, cloud-managed services)
- Streamline DevOps processes
- Deploy containers on IaaS or PaaS
- Leverage cloud-managed services

Level 3 -Cloud-Native Applications

- Transition to PaaS computing platforms
- Implement cloud-native applications and microservices architecture
- Foster long-term agility and scalability
- May require new code or adapting applications



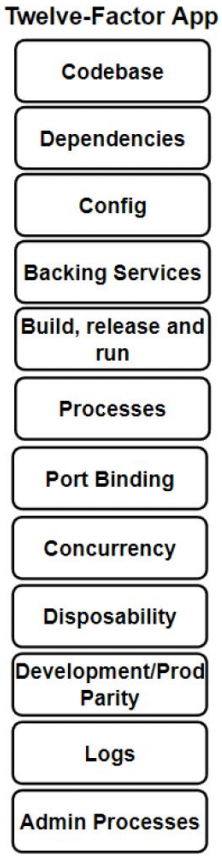
Cloud Native Fundamental Pattern & Principle

How would you design a Cloud-native architectures ? What are patterns, principles and best practices that we should follow when designing Cloud-native ?

Twelve-Factor Application principles:set of principles and practices that developers follow to create cloud-native applications.

What is The Twelve-Factor Application ?

- Methodology for building modern, scalable, and maintainable software-as-a-service applications.
- Created by Heroku, to provide a set of best practices and guidelines for designing and developing apps that can be easily deployed, scaled, and managed in the cloud.
- Designed to work well with modern cloud-native architectureslike microservices and container-based deployments.



Cloud native computing foundation (CNCF) - Trial Map

CNCF Cloud Native Trail Map

- Graphical guide created by the CNCF to help organizations and individuals navigate their cloud-native journey
- Recommended path for adopting cloud-native technologies, outlining the key steps, projects, and tools
- Useful starting point for those who are new to the cloud-native ecosystem or those looking to adopt cloud-native best practices.

Open Trail Map

https://raw.githubusercontent.com/cncf/trailmap/master/CNCF_TrailMap_latest.png

What are Containers

- Monolithic applications are deployed as a ***single unit*** and ***deploy whole application*** in one time.
- This caused temporarily un-available time of application, needs to rollback the whole deployment process.Can't split modules and scale independently.
- It solved in ***microservices*** architecture with ***Containers and Orchestrators.***
- Containers are ***package*** and ***distribute software applications***, makes them easy to deploy and run on different environments.
- Allow developers to package an application and its dependencies into a single, self-contained unit container images,easily shipped and run on any computer that has a container runtime.
- Containers are often used in the deployment of microservices,independent components can be developed, tested, and deployed separately.
- Each microservice is typically a self-contained unit, communicates with other microservices through well-defined interfaces.

Contd.

Containers provide to **decouple applications** with their own **os**, **dependencies** and **libraries**, perfect match to microservices deployments.

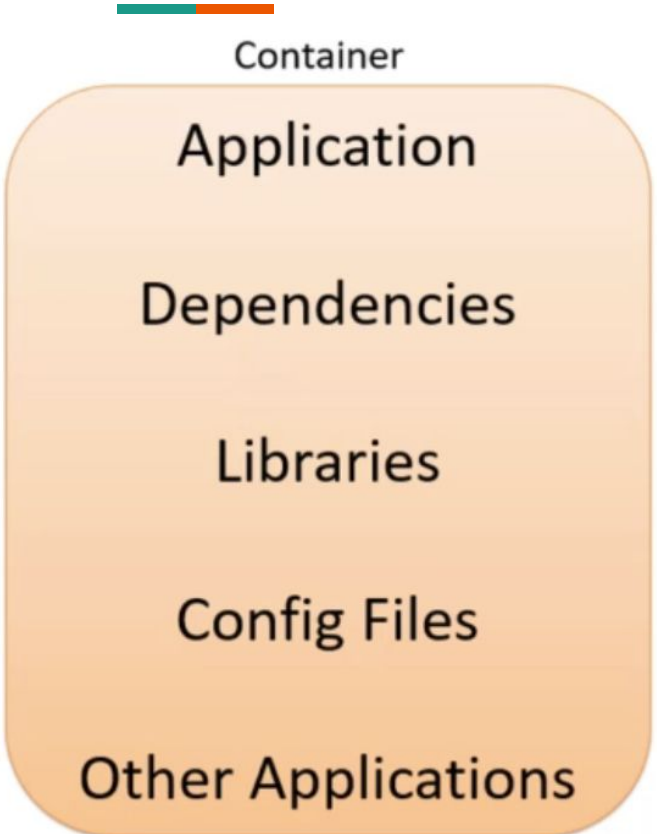
Microservices can **deployed separately** in a **container**. Each microservice can **deploy independently** with containers.

Since **deployed microservices separate container**, we can **scale** as per their **volume of traffics**.

Changes can be **applied independently** while other container stay not changed.

New features can be **applied** and **rollback** very easy with **container deployments.**

Docker is defacto standard for **containerization** of **microservices**.



- Container includes the entire runtime environment of the application and includes the application itself, all of its dependencies, all of its libraries, configuration files that are needed to run.
- Containerized application can move from platform to platform.
- Underlying infrastructure are abstracted and they're hidden from the application running within that container.

Advantages of Container

Isolation - Each microservice runs in its own container,provides isolation from the other microservices and the host operating system.

Scalability- Containers make it easy to scale microservices horizontallyby simply running more instances of a containerized microservice.

Portability - Containers allow microservices to be easily deployed and run on any computer or cloud platform that supports container runtime.

Resiliency - Microservices are intended to be independently deployable and scalable. Using containers in the deployment of microservices that can be quickly started and stopped, increase the resiliency of the application.

Best Practices for Containers

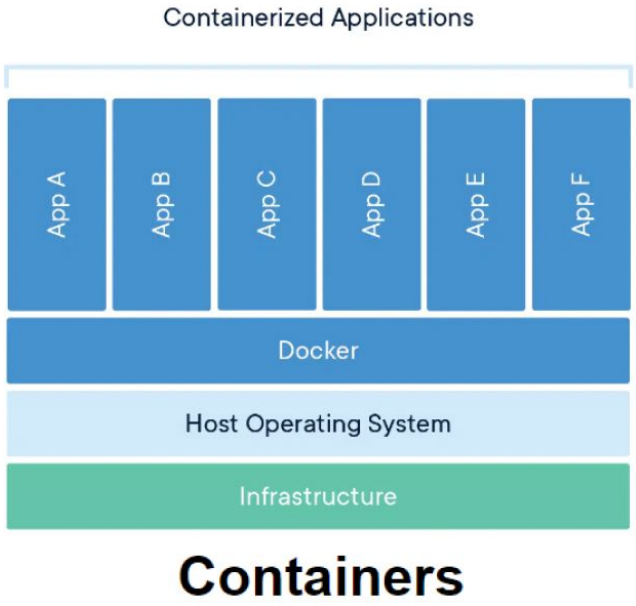
One Process per Container
- Follow single-responsibility principle
- Run only one primary process per container

Use Small Base Images
- Choose small, minimal base images (Alpine Linux, distroless)
- Reduces attack surface, build times, improves startup performance

Keep Containers Stateless
- Design containers to be stateless
- Easy replacement, scaling, updating without data loss
- Store stateful data in external services (databases, caches, object stores)

Use Environment Variables for Configuration
- Keep configuration settings external to the container
- Easy modification of settings without rebuilding container image

Label Container Images
- Use labels to provide metadata (version, build date, description)
- Helps with tracking, organization, auditing



Contd.

Version Container Images
- Use semantic versioning for container images
- Store them in a container registry

Implement Health Checks
- Add health checks to containers
- Allows orchestration system to restart or replace unhealthy instances

Use Container Orchestration
- Use orchestration platform (Kubernetes, Amazon ECS) for automation
- Simplifies management of complex applications and improves resilience

Monitor and Log
- Implement monitoring, logging, tracing for containerized services
- Use tools (Prometheus, Grafana, ELK stack) for metrics and logs

Continuous Integration and Deployment
- Automate building, testing, deploying with CI/CD pipelines
- Keeps application up-to-date, minimizes risk of human errors

How do Containers work

Containers are a form of lightweight virtualization.Allow multiple isolated apps to run on the same host.They share the host's kernel and resources while running in isolation.

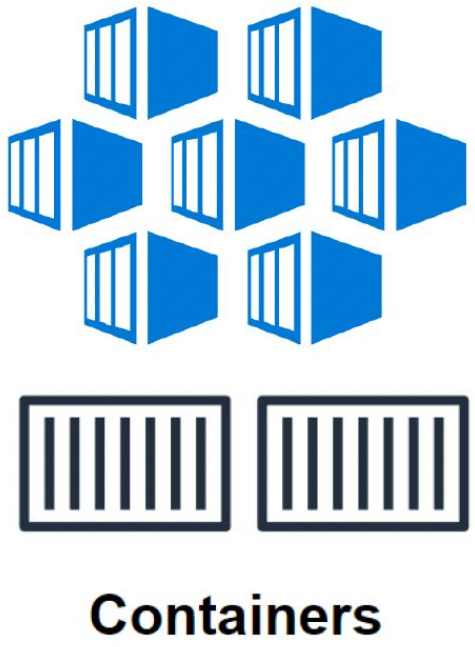
Container Image A self-contained package that includes the application, runtime, libraries, and settings.Constructed from a base image (usually a minimal OS) and a set of instructions.

Container Runtime Software that manages containers, providing an isolated environment and managing resources, lifecycle, and interactions with the host system.

Image Layers These are created for each instruction in a Dockerfile. Layers can be cached and reused, reducing build time and storage needs.

Namespaces Containers utilize Linux namespaces for process isolation. Creates an environment where a process can't see or interact with resources outside of its namespace.

Networking Containers communicate with each other and the host system through virtual network interfaces, which are created by container runtimes.

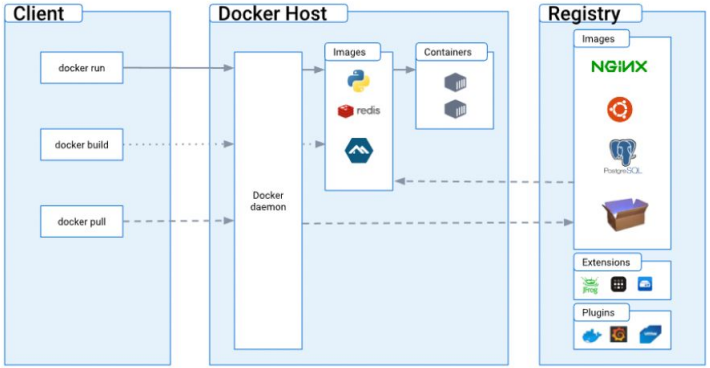


Running a Container

When a container runs, the **container runtime** creates an **isolated environment** using **namespaces**, **cgroups**, and a union file system.

The application starts within this environment, **utilizing the resources** and **dependencies** provided by the **container image**.

Despite **sharing the host's kernel** and **resources**, the container remains separate from other containers and the host itself.



Container Runtime

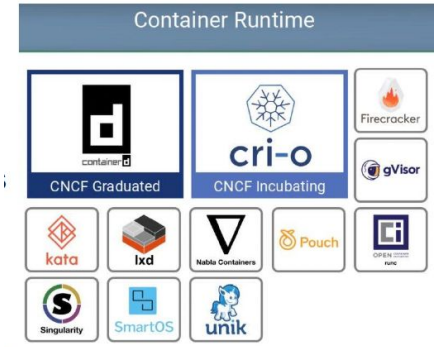
Container runtime executes and manages containers that provides the necessary environment to run containerized applications handles tasks; creation, starting, stopping, and monitoring of containers.

Docker The most known container runtime synonymous with container technology. Uses its own runtime, containerd, to manage containers.

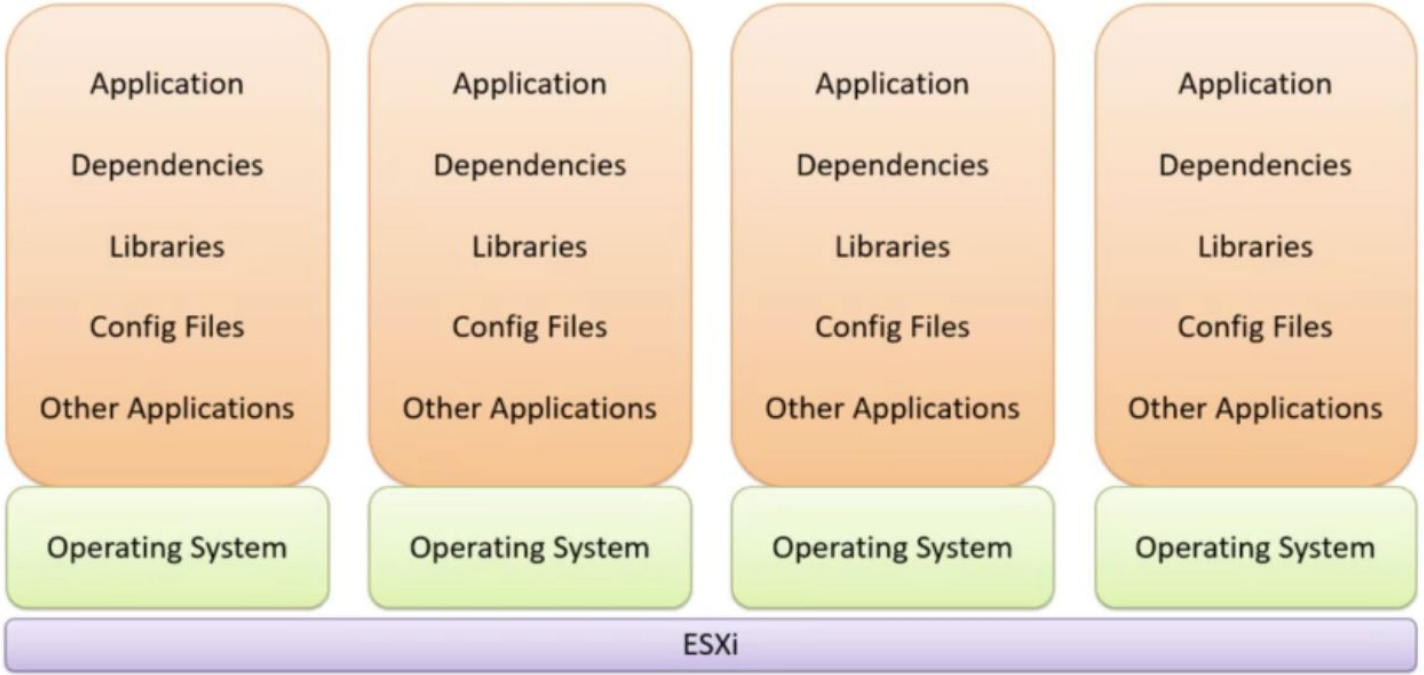
Containerd Docker's container runtime, also operable independently. It's lightweight and focuses on core container execution functionalities.

CRI-O Lightweight container runtime designed specifically for Kubernetes. Implements the Kubernetes Container Runtime Interface (CRI), allowing Kubernetes to use any OCI-compliant runtime.

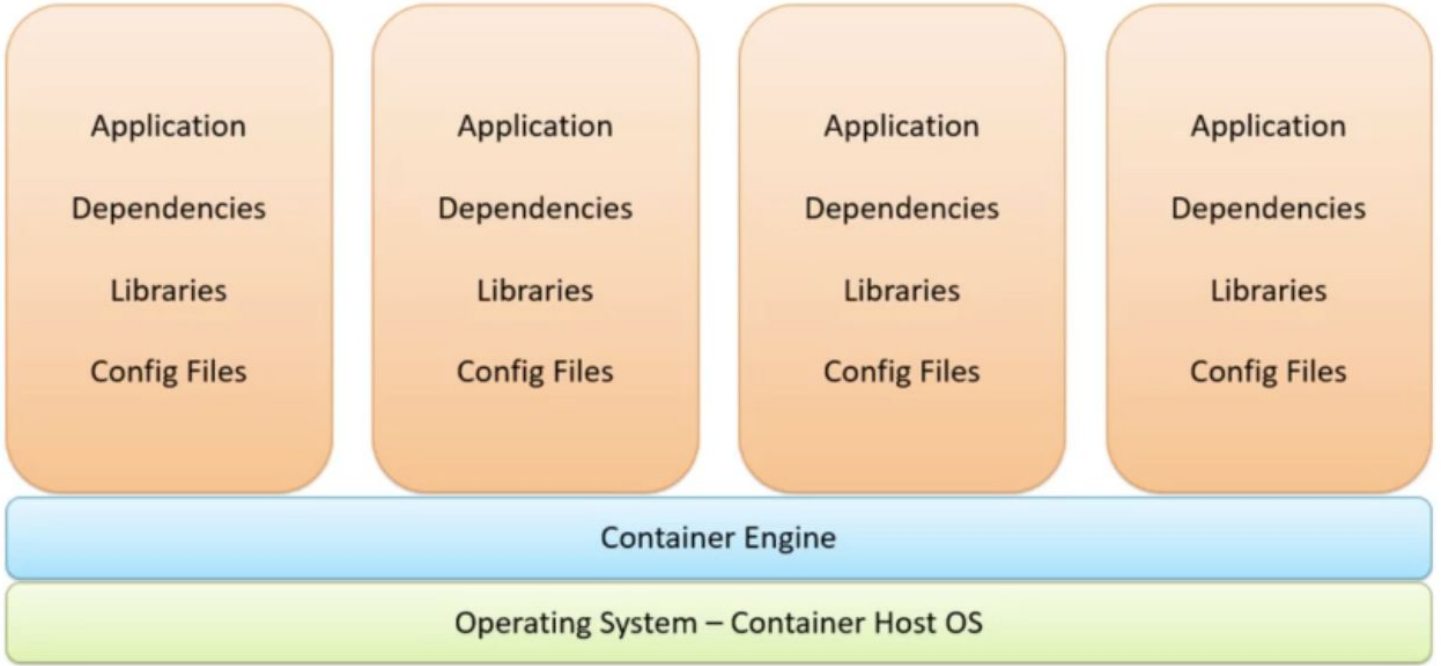
runc low-level container runtime based on the Open Container Initiative (OCI) runtime specification. It's the default runtime used by containerd and can also be used standalone for running containers.



Virtual Machines



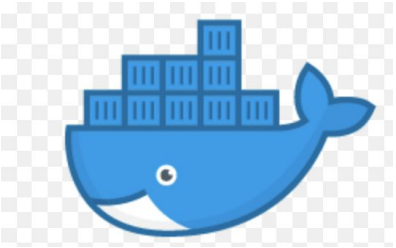
Contd

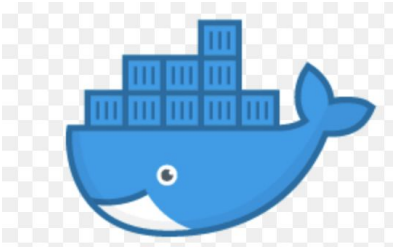

Application
Dependencies
Libraries
Config Files
Application
Dependencies
Libraries
Config Files
Application
Dependencies
Libraries
Config Files
Application
Dependencies
Libraries
Config Files
Container Engine
Operating System – Container Host OS

Introduction to Serverless

Virtual Machine

Container





Contd.

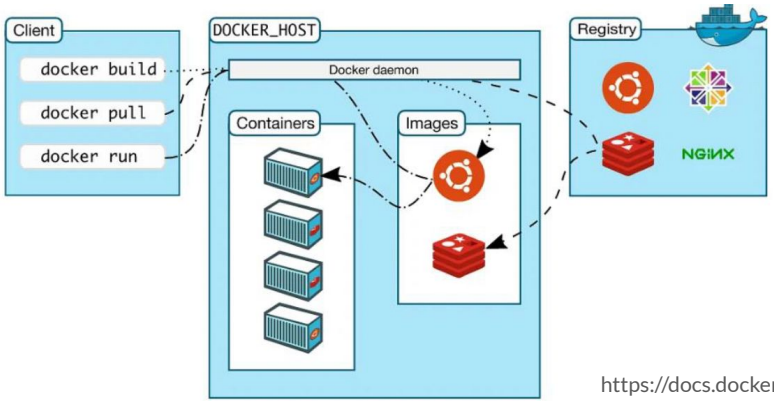
Vertical Scaling

Horizontal Scaling

Containerization, Running Microservices in Containers

Containerization is the process of **converting microservices** to **run** on **containers**.

- **Docker** is the most **popular platform** for **containerization**.
- The **code**, its **dependencies**, and **runtime** are **packaged** into a **container image**.
- **Images** are **stored** in a **container registry**, which acts like a repository.
- When needed, the image is transformed into a **running container instance** on any computer with a container runtime engine installed.



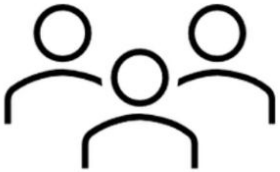
https://docs.docker.com/get-started/docker-overview/

Introduction

- Unification of Development & Operations
- Collaboration
- Dev-ops is a set of practices and a cultural movement
- Shorten development cycles
- Different organization have different standards and practices

Development Vs Operations

Development Team

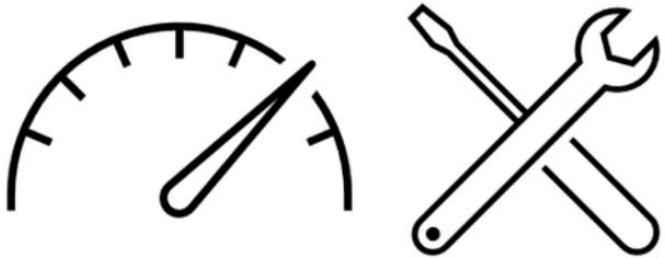


Operations Team

Objective: Speed!

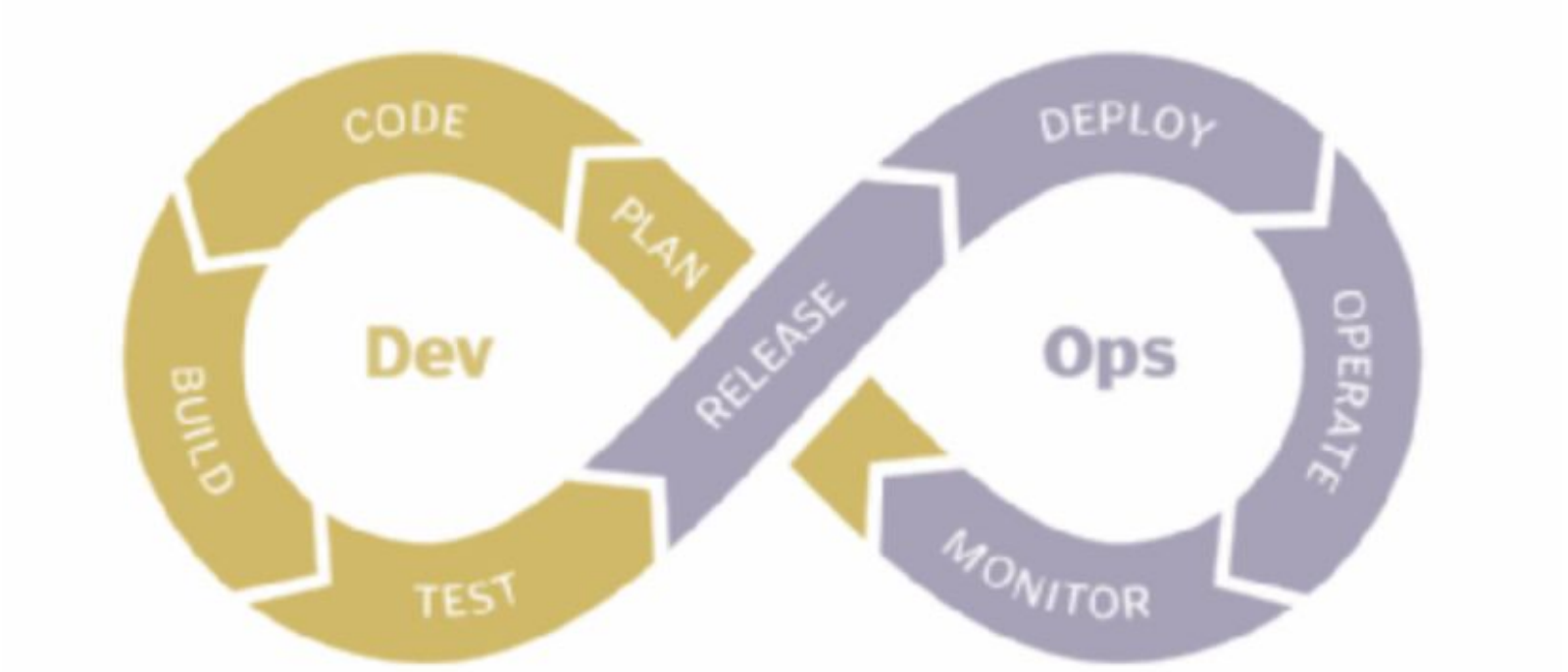
Objective: Stability!

DevOps Team



Objective: Speed and Stability!

DevOps loop


CODE
PLAN
BUILD
TEST
Dev
RELEASE
DEPLOY
OPERATE
MONITOR
Ops

Continuous Integration

Continuous integration is the practice of automating the integration of code changes from multiple contributors

- Developer pushes the code to a remote source control repository
  - Use Source control to keep track of changes
  - Merge code from more than one sources.
  - Push the code to cloud repository

- New code gets built in a pre-configured server.
- If the build succeeds, the server runs tests with that code.
- The tests pass a new code is added to the solution.

Contd

- Code is now in the cloud (GitHub, GitLab or Bitbucket etc). We require some server to connect to remote repository
- When connecting our remote repository to one of those services, we can specify what operating system we want our server to run.
- Configure the connection with server.
- The connection to the remote repository has two primary purposes.
    - The first purpose may be obvious after all the server needs the source code to build it.
        - Authenticate the server to access the code
    - The second purpose is what makes the integration of new code continuous.
        - Once the git pull or merge request is done, it notifies the server about new code being available for integration.

Instruction for integration can pre-configured or it can be provided inside the source code in the form of YAML Files

Contd.

In any case either pre-configured or because of the instructions, a server may then have to:

- Install some tools needed to compile your projects, SDKs mainly.
  - Often one can set precisely what version of the tools the server needs to install.
- A server may also need to: two, set some environment variables such as keys and passwords needed to build the code.
- Server needs to build the code using the operating system SDKs and versions previously specified.
- The server then needs to store the result of the build.
- The server needs to run some tests.
- The server will notify the remote repository about the results of build and test.

Test the Code

Two types of test that server may execute

- The first type is ***unit tests***.This type of test is performed right after the build succeeds as part of the continuous integration process.
- The server performs these unit tests, using code that developers have included in the source code.
- As with the builds, the server communicates the tests results back to the remote repository, so that developers are aware of any issues that may have arisen.

- The second type of test is the ***UI test***. The server executes these tests once it's compiled the code and often after the unit tests have passed.
- The UI tests exist to ensure the interface's quality and ensure that the screen elements respond to user interaction as they should. Such as how the application will look in diff screen sizes

Continuous Delivery

- Continuous Delivery is the ability to get changes of all types—including new features, configuration changes, bug fixes and experiments—into production, or into the hands of users, safely and quickly in a sustainable way.

- Goal is to make deployments—whether of a large-scale distributed system, a complex production environment, an embedded system, or an app—predictable, routine affairs that can be performed on demand.

- Two versions of CD process
  - A CD process that distributes code to Beta tester
  - A CD that distributes the new versions to all users once beta testers have given the green light.

Continuous Deployment

Continuous deployment goes one step further than continuous delivery. With this practice, every change that passes all stages of your production pipeline is released to your customers. There's no human intervention, and only a failed test will prevent a new change to be deployed to production.

Recap

Continuous Integration

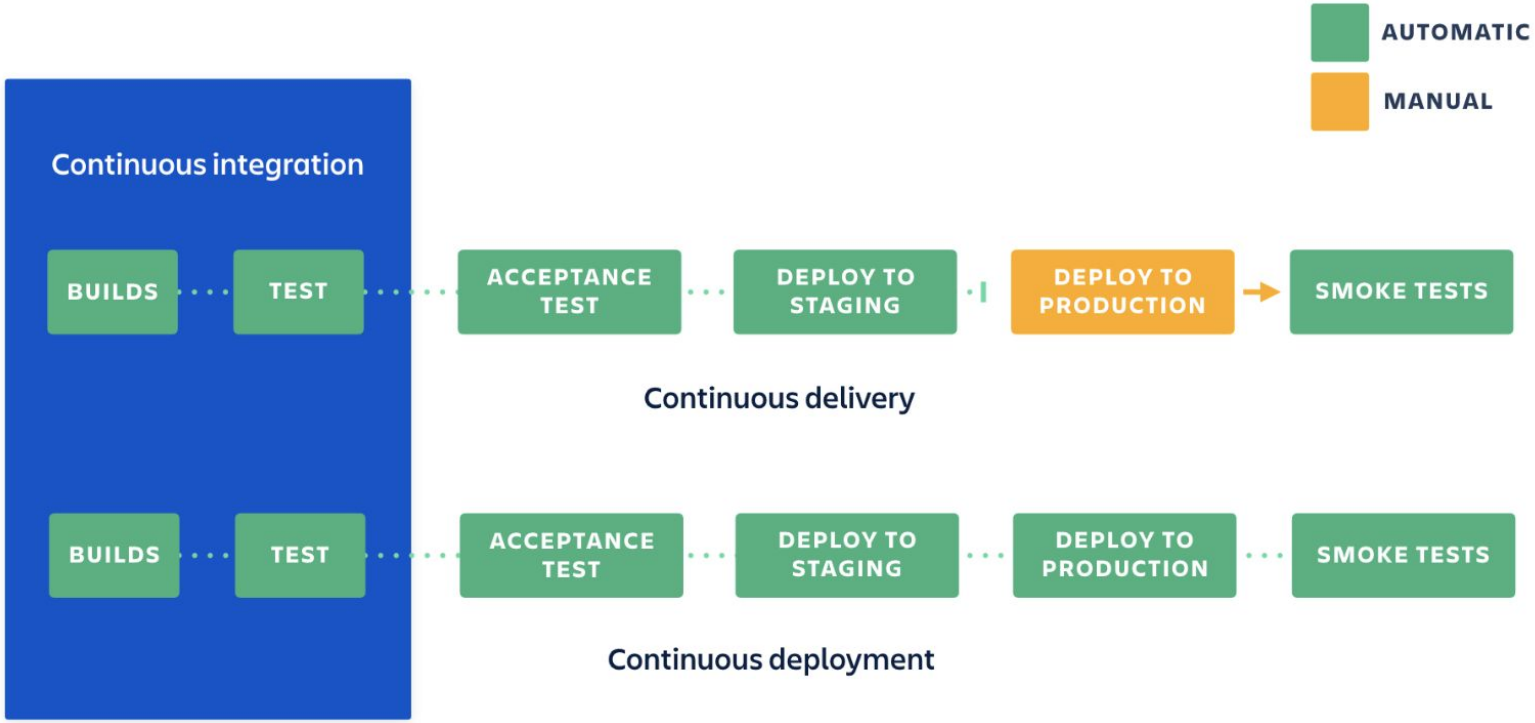
- Merging in small code changes frequently

Continuous Delivery

- Add additional automation and testing, get the code nearly ready to deploy with almost no human intervention

Continuous Deployment

- Deploying all the way into production without any human intervention


AUTOMATIC
MANUAL
Continuous integration
BUILDS
TEST
ACCEPTANCE TEST
DEPLOY TO STAGING
DEPLOY TO PRODUCTION
SMOKE TESTS
Continuous delivery
BUILDS
TEST
ACCEPTANCE TEST
DEPLOY TO STAGING
DEPLOY TO PRODUCTION
SMOKE TESTS
Continuous deployment

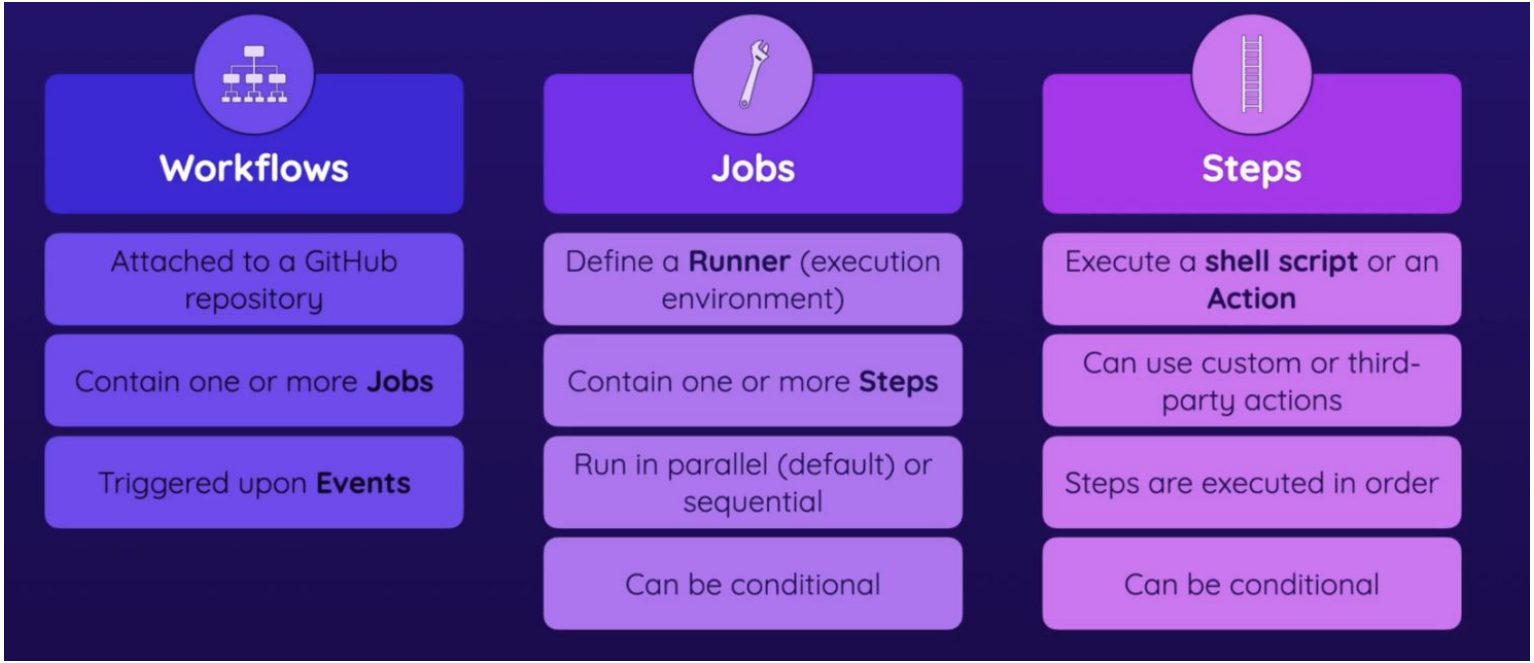
Tools - Continuous Integration

Github Actions

- Github Actions allows to build our code directly from GitHub and even integrate third-party actions from tools
- It allows access to Ubuntu,Windows and macOS servers to build our code and execute other command
- Free service allows to build 2000 minutes per month
- YAML file is used to execute Github Actions

```yaml
on:
  push:
    branches:
     - main
name: Continuous Integration process
jobs:
  build:
    name: Build App
    runs-on: ubuntu-latest
    steps:
      - run: flutter build
      - with:
        artifacts: "folder_path/file_name.aab"
```

Key Elements



Contd. Tools

Gitlab CI

- This is another CI tool and just like GitHub , it also build minutes which can be used for continuous integration and continuous delivery process.
- You could define the continuous integration and continuous delivery process with a YAML file similar to the one we saw earlier.

Jenkins

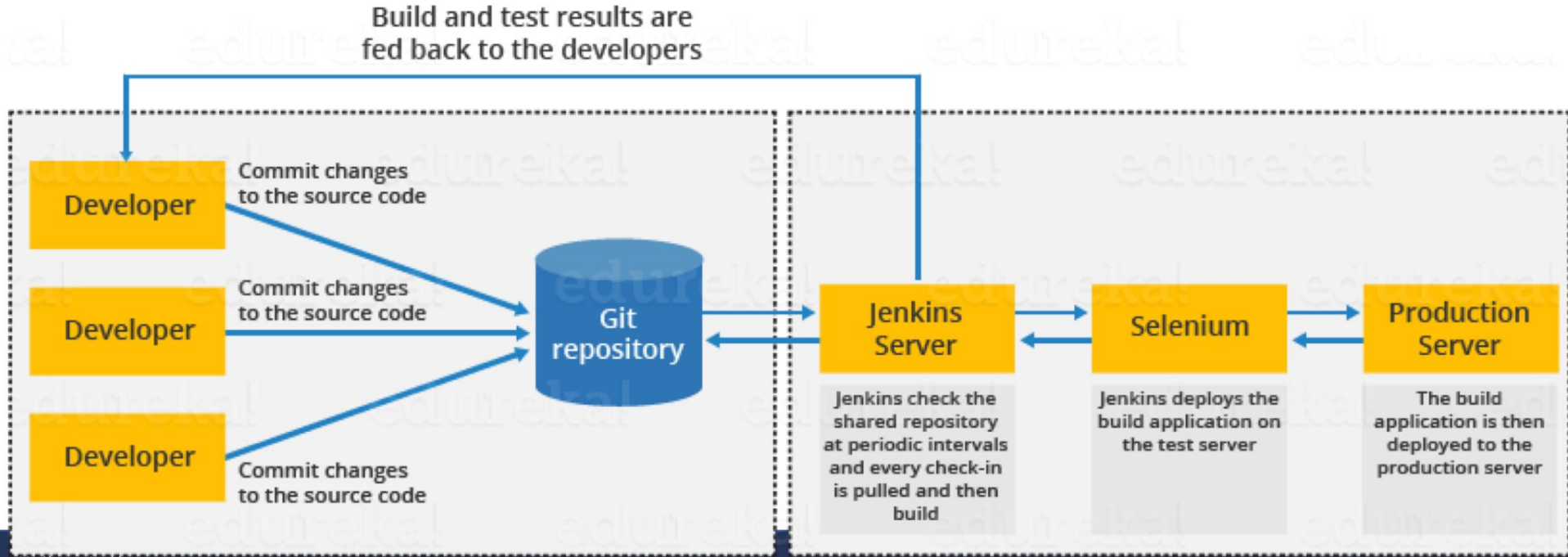
- Jenkins is perhaps the most popular continuous integration tool. It's open source and completely free.
- It allows for deep customizations of the continuous integration process.
- There are a wide variety of plugins readily available to facilitate your pipeline's configuration.
- However, using this tool requires a more hands-on approach to continuous integration, which has its benefits and disadvantages.

Contd

- Jenkins requires a dedicated server that would need to be maintained by the team while the other tools mentioned in this list are already cloud-based, so there's no need to maintain or scale your own servers in that scenario.

- Using cloud services from AWS or Azure can make Jenkins a bit easier though, providing your team with greater control at cheaper costs.

Tools - Continuous Integration



Tools - Continuous Deployment

AWS code deploy

- Most popular cloud services platform, providing a wide variety of cloud services such as code deploy etc.
- The code deploy service can automate your web project's deployment to other AWS such as EC2, AWS Fargate, AWS Lambda and even on-premises servers.
- Robust services , less downtime

Azure DevOps projects

- Similar to AWS , Azure provides a variety of services and the CD service is DevOps projects DevOps projects,
- which can help us run our production application on Windows or Linux servers and then deploy to other Azure services such as Azure Web Apps,virtual Machines or Service Fabric

Who Does Operations?

	Dev	Ops
Waterfall		Test, Staging, Production
Agile	Test	Staging, Production
DevOps	Test, Staging	Production
DistributedOps	Test, Staging, Production	Compliance and Guidance
NoOps	Test, Staging, Production	Compliance and Guidance

NETFLIX

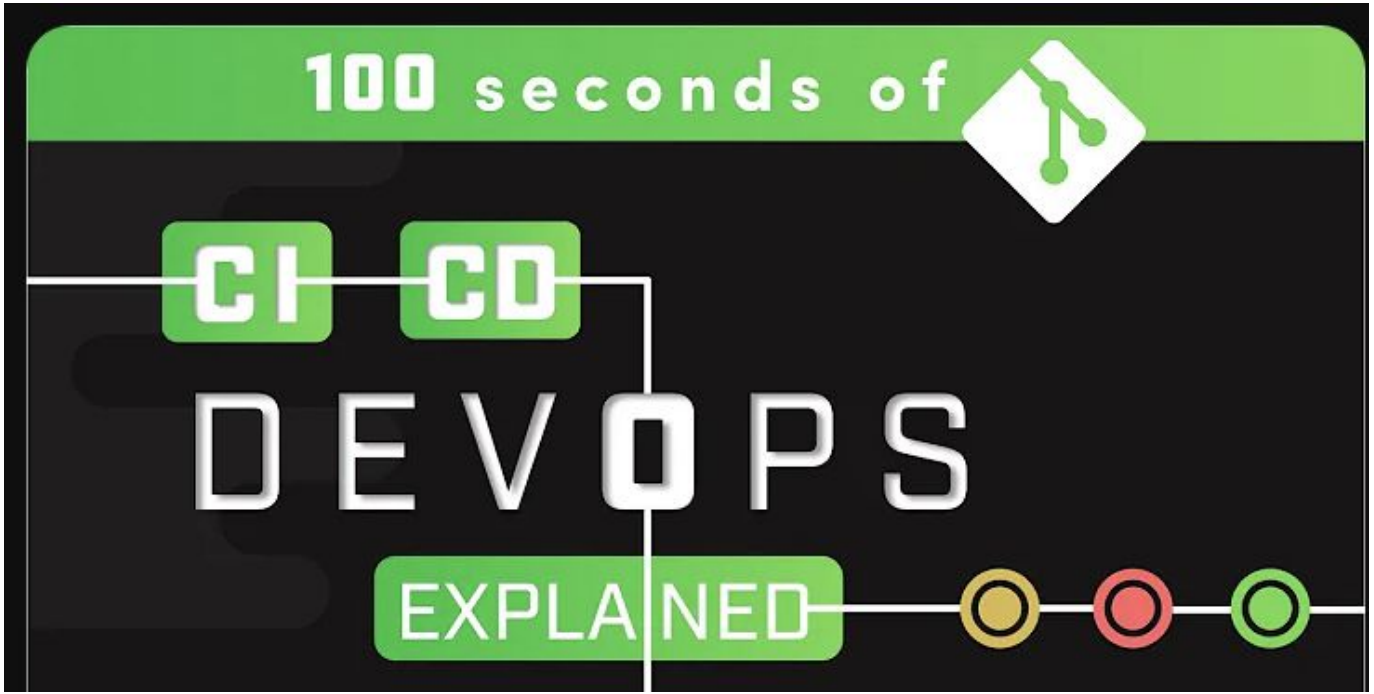
“Our journey to the cloud at Netflix began in August of 2008, when we experienced a major database corruption and for three days could not ship DVDs to our members. That is when we realised that we had to move away from vertically-scaled single points of failure, like relational databases in our datacenter, towards highly reliable, horizontally-scalable, distributed systems in the cloud.”

Yury Izrailevsky, VP, Cloud Computing and Platform Engineering, Netflix.

Netflix

- 60,000 configuration changes a day. 4000 commits a day.
- Every commit creates an Amazon Machine Imagine (AMI).
- AMI is automated deployed to a new RED/BLACK cluster.
- Have automated canary analysis, if okay, switch to new version, if not, rollback commit.


Time for DevOps



https://www.youtube.com/watch?v=scEDHsr3APg&ab_channel=Fireship

Thank you